Herald

Protocol Research: Agent Communication Protocol (IBM ACP) — and why Herald adopts A2A instead

Field	Value
Revision	1
Created	2026-05-22
Last modified	2026-05-22
Status	research-only — DOCUMENTING-WHY-REJECTED-AS-STANDALONE
Status summary	IBM's Agent Communication Protocol (ACP) launched May 2025 as a peer to A2A + MCP. On 2025-08-27, the i-am-bee/acp GitHub repository was archived with the announcement: "ACP is now part of A2A under the Linux Foundation." ACP exists only as historical context; Herald should adopt A2A (see A2A.md) and reference this doc to explain why ACP is not adopted as a standalone protocol. The architectural lessons from ACP (Semantic Layer, JSON-LD ontology, capability tokens, OTLP-first observability) inform Herald's A2A integration.
Issues	none — closed protocol; reference-only document.
Continuation	n/a — this protocol is superseded. The lessons inform A2A.md (Wave 4b).

Constitutional anchors

- **§107 anti-bluff** — this document does NOT propose any tests because Herald does not implement ACP. Anti-bluff applies to the A2A adoption (see A2A.md §5).
- **§11.4.74 catalogue-check** — n/a (not adopting).
- **§11.4.61 / branding** — tracked doc; regenerate siblings.
- **§106 spec-change** — no spec V3 amendment from this doc.

Table of contents

§1. Protocol overview

ACP — Agent Communication Protocol (sometimes "BeeAI ACP" to disambiguate from older usages). Launched by IBM in May 2025, a month after Google's A2A. Designed to standardize how AI agents communicate with each other and with infrastructure components, complementing MCP (which is "LLM ? tool") with an "agent ? agent" semantic vocabulary.

Wire format and architecture.

- **Transport:** HTTP/REST-based, with Server-Sent Events for streaming. Some sources describe it as JSON-RPC over HTTP/WebSockets; the canonical i-am-bee/acp repo uses OpenAPI 3.0 — REST endpoints — with JSON request/response bodies.
- **Brokered architecture:** three roles — Agent Clients, ACP Servers (registries), ACP Agents.
- **Semantic Layer:** a "JSON-LD ontology of intent" defining universal verbs (`QUERY`, `EXECUTE`, `DELEGATE`, `NEGOTIATE`).
- **Multipart messages:** `Message` object contains ordered `MessagePart` components supporting multimodal content (text, images, JSON).

Reference implementation. BeeAI Framework — Python + TypeScript SDKs. The BeeAI Platform (`localhost:8333` web UI; `beeai run` CLI). **No Go SDK** was produced before the project archived.

Authentication. "Capability tokens" — unforgeable signed objects encoding `(resource type, ops, expiry)`. The protocol also bridged Kubernetes RBAC to leverage existing cluster roles. Auth design differed from A2A's OAuth-first approach.

Observability. OTLP-first — ACP calls were instrumented with OpenTelemetry by default; BeeAI shipped traces to Arize Phoenix out of the box.

Governance. Initially developed under the Linux Foundation AI & Data program for open governance.

Versioning. ACP releases ran from v0.x.x through v1.0.3 (released 2025-08-21). The project was then archived on 2025-08-27.

§2. Specification deep-dive

(Documented for historical / educational completeness only. Implementers should NOT use this as a build target.)

§2.1 Message envelope

```
{
  "id": "uuid",
  "role": "user" | "agent",
  "parts": [
    { "content_type": "text/plain", "content": "..." },
    { "content_type": "application/json", "content": { "...":
        "..." } },
```

```
    { "content_type": "image/png", "content_url": "..." }
  ],
  "metadata": { "..." }
}
```

§2.2 Endpoints (OpenAPI)

- `POST /agents/{agent_id}/messages` — send a message; returns task ID.
- `GET /agents/{agent_id}/messages/{message_id}` — fetch status.
- `GET /agents/{agent_id}/messages/{message_id}/stream` — SSE streaming.
- `GET /agents/{agent_id}/manifest` — fetch agent capability manifest.

§2.3 Agent manifest

Similar to A2A's AgentCard but with JSON-LD typing:

```
{
  "@context": "https://acp.dev/contexts/agent.jsonld",
  "@type": "Agent",
  "name": "...",
  "description": "...",
  "capabilities": [
    { "@type": "Capability", "verb": "QUERY", "resource": "..." }
  ]
}
```

§2.4 Sync vs async vs streaming

- **Sync:** HTTP POST returning JSON response (short tasks).
- **Async:** fire-and-forget with task ID; client polls or subscribes for progress.
- **Streaming:** server pushes incremental delta messages over WebSockets/SSE.

§2.5 Capability tokens (auth)

Capability tokens were ACP's distinguishing security mechanism — JWS-signed objects encoding what a holder is authorized to do. The tokens were verifiable by any agent without contacting an issuer, similar to macaroons.

§2.6 Compared to A2A and MCP (at launch, May 2025)

Aspect	ACP (IBM)	A2A (Google)	MCP (Anthropic)
Focus	agent-to-agent messaging	cross-vendor agent interop	LLM-to-tool
Default transport	REST + SSE	JSON-RPC + SSE	stdio + Streamable HTTP
Auth	Capability tokens	OAuth 2.0 / mTLS / Signed cards	OAuth 2.0 / stdio trust
Discovery			configured by host

Aspect	ACP (IBM)	A2A (Google)	MCP (Anthropic)
	ACP Servers (registries)	`/.well-known/agent.json`	
Topology	brokered (registry-mediated)	peer-to-peer	host-server
Multimodal	JSON-LD-typed MessageParts	typed Parts (text/file/data)	resource MIME types
Reference SDK	BeeAI (Python, TypeScript)	a2a-* SDKs (8 languages incl. Go)	go-sdk + Python/TS/Java/C#/Rust/Swift
Governance	Linux Foundation AI & Data	Linux Foundation (A2A project)	open-source, MCP foundation forming

§3. The 2025-08-27 merger into A2A

On 2025-08-27, the `i-am-bee/acp` GitHub repository was archived. The README and project announcements stated unambiguously:

> "ACP is now part of A2A under the Linux Foundation."

What this means in practice:

- The ACP repo is read-only as of 2025-08-27; no new releases.
- ACP's architectural concepts (Semantic Layer, capability tokens, brokered registries) are being absorbed into A2A as **extensions** or as inspiration for v1.x features.
- BeeAI Framework remains active but pivots toward A2A as its primary protocol — see the BeeAI A2A integration docs at https://framework.beeai.dev/integrations/acp.
- New projects choosing between ACP and A2A are explicitly directed to A2A by the Linux Foundation.

Why the merger? The agentic protocol ecosystem fragmented in early-to-mid 2025: MCP (Anthropic), A2A (Google), ACP (IBM), ANP (community). The cost of supporting all four (in SDKs, in agent runtimes, in interop testing) was unsustainable. The Linux Foundation brokered consolidation — A2A absorbed ACP because A2A had broader ecosystem support (Microsoft, AWS, Salesforce, SAP, ServiceNow joined A2A by mid-2025; ACP did not match that adoption).

Where ACP ideas live now in A2A:

- **AgentCard.skills[].tags** — partially absorbed ACP's JSON-LD-typed capabilities.
- **Signed AgentCards (A2A v1.0)** — directly inspired by ACP capability tokens.
- **Push notifications** — A2A's webhook-based async completion echoes ACP's async pattern.
- **Streaming via SSE** — both protocols converged on the same pattern.

§4. Lessons Herald carries forward to A2A

When Herald implements A2A (per A2A.md, Wave 4b), the following ACP design choices inform Herald-specific extensions:

§4.1 OTLP-first observability

ACP shipped OTel by default. Herald MUST do the same on its A2A surface — `traceparent` propagation on every A2A request, with trace export to Herald's existing OTel collector (from `submodules/observability/`). This is already in §3.7 of A2A.md.

§4.2 Capability tokens as a future extension

A2A's Signed AgentCards cover authentication of the CARD. ACP's capability tokens covered fine-grained authorization of EACH REQUEST. Herald could layer ACP-style capability tokens ON TOP of A2A as a `commons_a2a/capabilities/` extension — see HRD-242 follow-up.

§4.3 Multimodal MessageParts

Both ACP and A2A support multimodal MessageParts. Herald's existing `commons.OutboundMessage.Body` type carries multimodal content (text + markdown + HTML + attachments). The translation between A2A Parts and Herald's Body is straightforward; the ACP multimodal pattern validated the approach.

§4.4 Brokered registry as a future pattern

ACP's "ACP Servers (registries)" is a brokered discovery layer. Herald could publish AgentCards to a Linux Foundation A2A registry — see HRD-247 follow-up.

§4.5 JSON-LD typed capabilities

ACP used JSON-LD `@context` for capability typing. A2A uses untyped `skills[].tags`. Herald can adopt a hybrid — publish A2A skills with `tags` AND a Herald-specific `metadata.jsonld_context` for consumers that want strict typing. This is a Wave 4b+1 enhancement.

§5. Why Herald does NOT implement ACP separately

Six reasons:

1. **The protocol is archived.** No upstream maintenance; security patches go unfixed; ecosystem alignment shifts toward A2A.
2. **No Go SDK.** Implementing ACP would mean writing a Go client from scratch — wasted effort vs adopting the actively-maintained a2a-go SDK.
3. **Overlapping use cases.** Every ACP use case has an A2A equivalent. Building both surfaces is duplicated work.
4. **BeeAI Framework itself pivoted to A2A.** The canonical ACP runtime now speaks A2A as primary.
5. **Operator mandate mentioned "ACP — for work with models (LLMs)".** The operator likely conflated ACP and A2A (a common confusion in 2025-2026). The right semantic match for "agent communication for LLMs" in 2026 is A2A. This doc is the disambiguation record.
6. **Herald is small.** Herald supports 7 flavor binaries + 1 future Wave 4 protocol surface; adding a deprecated protocol is non-trivial overhead with zero payoff.

Decision (operator confirmation pending): Herald implements A2A (per A2A.md, Wave 4b) AS THE response to the operator's "ACP — for work with models (LLMs)" mandate. This doc serves as the forensic trail explaining why "implement ACP" became "implement A2A".

§6. Historical references

(All fetched 2026-05-22; historical record.)

- **IBM Research project page.** https://research.ibm.com/projects/agent-communication-protocol
- **IBM Think topic page.** https://www.ibm.com/think/topics/agent-communication-protocol
- **i-am-bee/acp repository.** https://github.com/i-am-bee/acp — ARCHIVED 2025-08-27, v1.0.3 final release 2025-08-21. 1k+ stars, 119 forks. Python 77.9%, TypeScript 20.9%.
- **BeeAI Framework ACP integration (post-merger).** https://framework.beeai.dev/integrations/acp
- **BeeAI pre-alpha SDK docs.** https://docs.beeai.dev/acp/pre-alpha/sdk
- **WorkOS technical overview.** https://workos.com/blog/ibm-agent-communication-protocol-acp — comprehensive technical breakdown including the architecture comparison with MCP and A2A.
- **Medium introduction.** https://medium.com/mitb-for-all/introducing-the-agent-communication-protocol-acp-abd882114139
- **Linux Foundation AI & Data announcement** (referenced from i-am-bee/acp README).
- **Survey arxiv:2505.02279** — published May 2025, before the merger; documents ACP/A2A/MCP/ANP as four-way separate. The post-merger landscape is now A2A/MCP (+ ANP as community alternative).
- **arXiv:2602.15055** — "Beyond Context Sharing: A Unified Agent Communication Protocol" — post-merger analysis (Feb 2026) arguing the consolidation was correct.